

Default Conventions in Entity Framework Core

In this article, I will discuss **Default Conventions in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [Explicit Loading in Entity Framework \(EF\) Core](#) with Examples.

What are Default Conventions in EF Core?

In Entity Framework Core (EF Core), default conventions are predefined rules that EF Core uses to determine how entity classes map to database tables and columns. These conventions enable EF Core to automatically generate database schemas without requiring explicit configuration. This “Convention over Configuration” approach simplifies development by automating tasks like:

- Automatically detect Primary Key and Foreign Key properties based on naming conventions.
- Setting up one-to-one, one-to-many, and many-to-many Relationships Between Tables based on the navigation properties defined in the entities.
- Determining Column Data Types based on the .NET data types.
- Determining the Table and Column Names based on default naming conventions.
- Configure columns as nullable or non-nullable based on the .NET type's nullability.
- Automatically generate indexes and constraints for keys and foreign keys. For primary key clustered index and for foreign key non-clustered index.
- Automatically Set up the default cascade delete behaviors based on the relationship types.

While Default Conventions offer a solid starting point, they can be overridden using Data Annotations or the Fluent API for custom configurations, which we will discuss in our upcoming sessions. In this session, let's focus on understanding the default conventions in detail.

Example to Understand Default Conventions in Entity Framework Core:

To illustrate EF Core's Default Conventions, let's build a real-world example using a student management system in a .NET console application. We will define the following entities:

- Student
- Teacher
- Course
- Address
- Gender (Enumeration)

Then, we will see how EF Core automatically configures database schema such as table names, column names, primary keys (PK), foreign keys (FK), indexes, column data types, column attributes (e.g., nullable, not nullable, identity), relationships between tables, and cascade behaviors.

Create Model Classes

Let's start by creating the following model classes. Create a folder called **Entities**, and inside that folder, please create the following models.

Student

Create a class file named **Student.cs** within the Entities folder and then copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
```

```

    public class Student
    {
        public int StudentId { get; set; } // SQL Type: INT (NOT NULL,
Primary Key)
        public string FirstName { get; set; } // SQL Type: NVARCHAR(MAX)
(NOT NULL)
        public string? LastName { get; set; } // SQL Type: NVARCHAR(MAX)
(NULL)
        public DateTime? DateOfBirth { get; set; } // SQL Type: DATETIME2
(NULL)
        public decimal GPA { get; set; } // SQL Type: DECIMAL(18, 2) (NOT
NULL)
        public bool IsActive { get; set; } // SQL Type: BIT (NOT NULL)
        public byte[] ProfilePicture { get; set; } // SQL Type:
VARBINARY(MAX) (NOT NULL)
        public virtual Gender Gender { get; set; } // SQL Type: INT (NOT
NULL, because enums are stored as INT)
        public virtual Address Address { get; set; } // SQL Type: This would
create a foreign key with default INT (if Address is required, it would be
NOT NULL)
        public virtual ICollection<Course> Courses { get; set; } // SQL
Type: This would create a join table for many-to-many relationships
    }
}

```

Key Points:

- **Primary Key:** EF Core recognizes the StudentId property as the primary key due to its naming convention.
- **Nullable Properties:** LastName and DateOfBirth are nullable, allowing NULL values in the database.
- **Enum Mapping:** The Gender property is an enumeration stored as an INT in the database.
- **Navigation Properties:** Address and Courses establish relationships with other entities.

Teacher

Create a class file named **Teacher.cs** within the Entities folder and then copy and paste the following code.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Teacher
    {
        public int TeacherId { get; set; } // PK (INT, Identity)
        public string FullName { get; set; } // NVARCHAR(MAX), NOT NULL
        public DateTime HireDate { get; set; } // DateTime, NOT NULL
        public TimeSpan WorkHours { get; set; } // TIME, NOT NULL
        public decimal Salary { get; set; } // DECIMAL(18,2), NOT NULL
        public bool IsTenured { get; set; } // BIT, NOT NULL
        public virtual ICollection<Course> Courses { get; set; } // One-to-
Many relationship
    }
}

```

Key Points:

- **Primary Key:** TeacherId is automatically identified as the primary key.
- **Navigation Property:** Courses establish a one-to-many relationship with the Course entity.

Course

Create a class file named **Course.cs** within the Entities folder, and then copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Course
    {
        public int CourseId { get; set; } // PK (INT, Identity)
        public string Title { get; set; } // NVARCHAR(MAX), NOT NULL
        public double Credits { get; set; } // FLOAT, NOT NULL
        public int TeacherId { get; set; } // FK to Teacher (INT, NOT NULL)
        public virtual Teacher Teacher { get; set; } // Navigation property
        public virtual ICollection<Student> Students { get; set; } // Many-
to-Many relationship
    }
}
```

Key Points:

- **Primary Key:** CourseId is automatically identified as the primary key.
- **Foreign Key:** TeacherId links the Course to a Teacher.
- **Navigation Properties:** Teacher and Students define relationships with Teacher and Student entities, respectively.

Address

Create a class file named **Address.cs** within the Entities folder and then copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Address
    {
        public string Street { get; set; }
        public string City { get; set; }
        public string PostalCode { get; set; }
        public int AddressId { get; set; }
        public string? State { get; set; }
        public string Country { get; set; }
        public int? StudentId { get; set; }
        public virtual Student Student { get; set; }
        public int? TeacherId { get; set; }
        public virtual Teacher Teacher { get; set; }
    }
}
```

Key Points:

- **Primary Key:** AddressId is the primary key.
- **Foreign Keys:** StudentId and TeacherId link Address to Student and Teacher entities, respectively.
- **Optional Relationships:** Both foreign keys are nullable, indicating that an address may belong to a student, a teacher, or neither.

Gender

Create a class file named **Gender.cs** within the Entities folder, and then copy and paste the following code. This is going to be an Enum to hold the Gender-named constants. By default, EF Core stores enums as their underlying integer values in the database.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public enum Gender
    {
        Male = 1,
        Female = 2,
        Other = 3
    }
}
```

Modifying DbContext Class

The DbContext is the bridge between domain classes and the database. It manages entity configurations, connections, and more. Next, please modify the **EFCoreDbContext** class as follows. Here, we are adding the Entities as DbSet properties.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Connection String
            optionsBuilder.UseSqlServer("Server=LAPTOP-6P5NK25R\\SQLSERVER2022DEV;Database=MyStudentDB;Trusted_Connection=True;TrustServerCertificate=True");
            // DbSet properties represent the tables in the database.
            // Each DbSet corresponds to a table, and the type parameter corresponds to that table.
            public DbSet<Student> Students { get; set; }
            public DbSet<Teacher> Teachers { get; set; }
            public DbSet<Course> Courses { get; set; }
            public DbSet<Address> Addresses { get; set; }
        }
    }
}
```

Key Points:

- **Connection String:** Specifies the database server and database name. In production applications, it's best practice to store connection strings securely, such as in configuration files or environment variables.
- **DbSet Properties:** Each `DbSet<TEntity>` corresponds to a table in the database and allows CRUD operations on the entities.

Generating and Applying Migrations

Migrations in EF Core allow us to update the database schema to match domain models while preserving existing data. So, after setting up the models and DbContext, we need to create the migration and apply it to sync our models with the database schema.

So, open the Package Manager Console and execute the **Add-Migration ExtendingModel** command as follows. You can give your migration any name. I am giving the name `ExtendingModel`.

```
Package Manager Console
Package source: All Default project: EFCoreCodeFirstDemo
PM> Add-Migration ExtendingModel
Build started...
Build succeeded.
No store type was specified for the decimal property 'GPA' on entity type 'Student'. This will cause
values to be silently truncated if they do not fit in the default precision and scale. Explicitly specify
the SQL server column type that can accommodate all the values in 'OnModelCreating' using 'HasColumnType',
specify precision and scale using 'HasPrecision', or configure a value converter using 'HasConversion'.
No store type was specified for the decimal property 'Salary' on entity type 'Teacher'. This will cause
values to be silently truncated if they do not fit in the default precision and scale. Explicitly specify
the SQL server column type that can accommodate all the values in 'OnModelCreating' using 'HasColumnType',
specify precision and scale using 'HasPrecision', or configure a value converter using 'HasConversion'.
To undo this action, use Remove-Migration.
```

Warning for Decimal Property Precision:

By default, EF Core maps decimal properties to the SQL data type **DECIMAL (18,2)**. The database column can store up to 18 digits, with two digits after the decimal point. If the values you want to store in the GPA or Salary property require more precision (e.g., more than 2 decimal places), those values will be silently truncated, causing a loss of precision. That is the reason why we are getting the above warning. In our upcoming session, we will see how to overcome these warnings using Data annotation and Fluent API configurations. For now, ignore the warning.

Updating the Database:

Again, open the Package Manager Console and execute the **Update-Database** command as follows:

```
Package Manager Console
Package source: All Default project: EFCoreCodeFirstDemo
PM> Update-Database
Build started...
Build succeeded.
No store type was specified for the decimal property 'GPA' on entity type 'Student'. This will cause
values to be silently truncated if they do not fit in the default precision and scale. Explicitly specify
the SQL server column type that can accommodate all the values in 'OnModelCreating' using 'HasColumnType',
specify precision and scale using 'HasPrecision', or configure a value converter using 'HasConversion'.
No store type was specified for the decimal property 'Salary' on entity type 'Teacher'. This will cause
values to be silently truncated if they do not fit in the default precision and scale. Explicitly specify
the SQL server column type that can accommodate all the values in 'OnModelCreating' using 'HasColumnType',
specify precision and scale using 'HasPrecision', or configure a value converter using 'HasConversion'.
Applying migration '20240908142246_ExtendingModel'.
```

This command applies the pending migrations to the database, creating the necessary tables and schema based on your models. Upon successful execution, EF Core creates the MyStudentDB database with the configured tables.

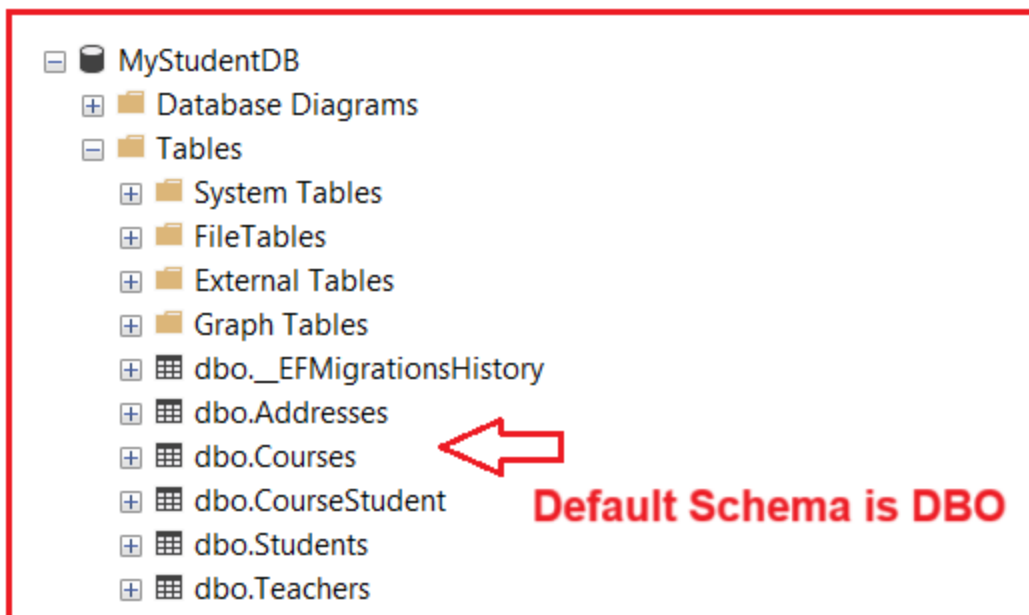
What are the Default Entity Framework Core Conventions?

EF Core's Default Conventions cover a wide range of configurations. Let us understand these Default Entity Framework Core Conventions one by one.

Default Schema in EF Core:

EF Core uses the database provider's default schema. For SQL Server, this is typically dbo. You can also specify a different schema using the Fluent API or Data Annotations if needed, which we will discuss in our coming sessions. For example, if you have a Student entity, EF Core will create the table as dbo.Students.

Now, if you verify the MyStudentDB database tables, you will see that they are created with the dbo schema, as shown in the image below.



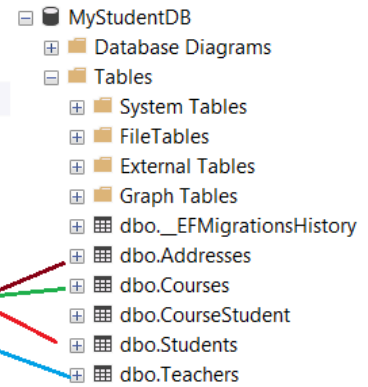
Default Table Name in EF Core:

Tables are named after the DbSet<TEntity> properties in the DbContext. If a DbSet is named Students, the corresponding table will be Students. EF Core does not automatically pluralize table names. It uses the exact name provided in the DbSet or the class name if DbSet is not specified. For example, **public DbSet<Student> Students** results in a table named **Students**. For a better understanding, please have a look at the following image.

```

public class EFCoreDbContext : DbContext
{
    0 references
    protected override void OnConfiguring(DbContextOptionsBuilder
    0 references
    public DbSet<Student> Students { get; set; }
    0 references
    public DbSet<Teacher> Teachers { get; set; }
    0 references
    public DbSet<Course> Courses { get; set; }
    0 references
    public DbSet<Address> Addresses { get; set; }
}

```

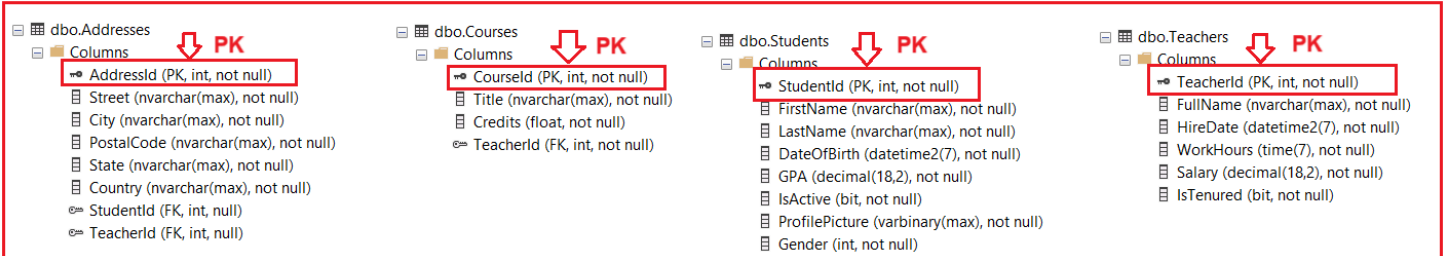


Default Primary Key Name in EF Core:

EF Core identifies a property as a primary key if it follows the naming conventions:

- A property named `Id`.
- A property named `<EntityName>Id` (e.g., `StudentId`).

The primary key is automatically configured as NOT NULL and set as the table's primary key constraint. If both `Id` and `<EntityName>Id` are present, `Id` takes precedence. By default, the primary key column is placed first in the table. The absence of a key property results in an exception during migration. For a better understanding, please have a look at the following diagram.



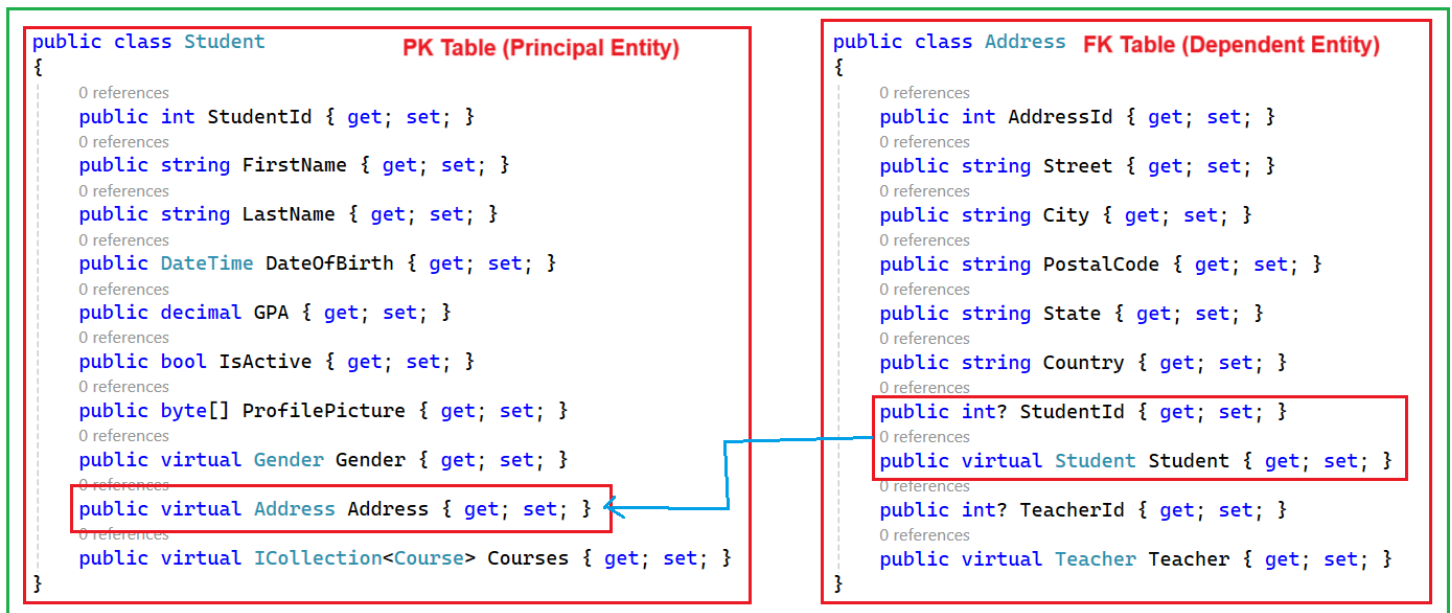
Default Foreign Key Column Name in EF Core:

In Entity Framework Core (EF Core), the default foreign key column name is determined based on the relationship between the entities.

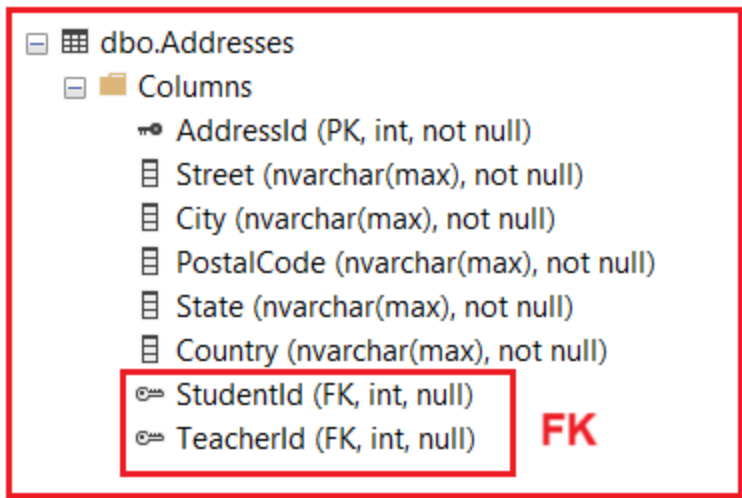
Single Foreign Key (One-to-Many or One-to-One):

By default, the FK column is named `<RelatedEntityName>Id`. For example, the Address table has a foreign key named `StudentId` referencing the Student table. In the Courses table, `TeacherId` is the foreign key to the Teachers table.

EF Core Automatically configures the foreign key relationships based on navigation properties and matching key properties. For a better understanding, please have a look at the following two entities.

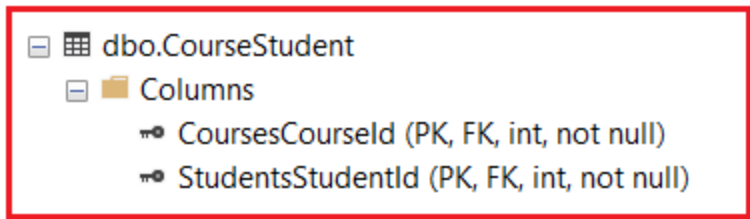


In this case, EF Core will automatically name the foreign key column as StudentId in the Addresses table. For a better understanding, please check the Addresses table, and you should see the following:



Composite Foreign Key (Many-to-Many Relationship):

For many-to-many relationships, EF Core creates a junction (or join) table with foreign key columns referring to both related entities. The foreign key columns will be named based on the format <Table1Name>PrimaryKey and <Table2Name>PrimaryKey. For a many-to-many relationship between Student and Course, EF Core will generate a junction table (CourseStudent) with two foreign key columns as shown in the below image:

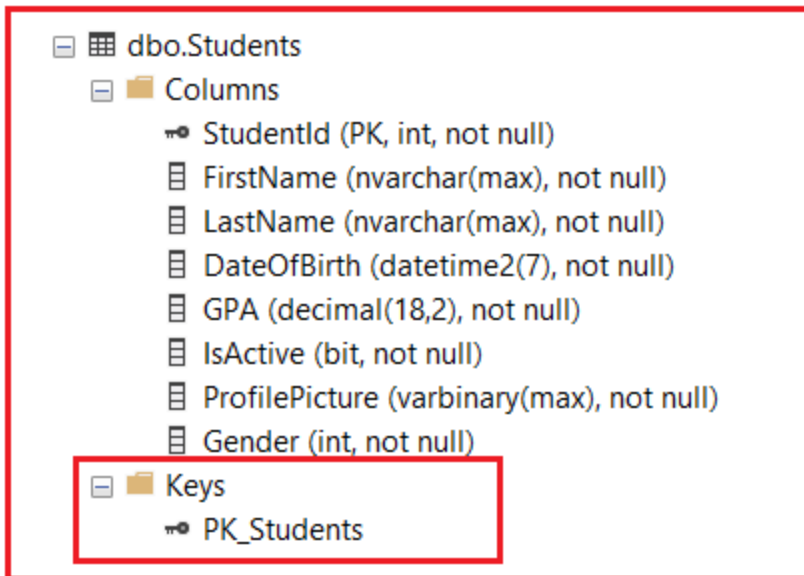


Default Constraints in EF Core:

In Entity Framework Core (EF Core), several types of constraints are automatically generated when creating database tables. These include primary key constraints, foreign key constraints, and unique constraints. These constraints can be customized using the Fluent API or Data Annotations if specific naming or behaviors are required.

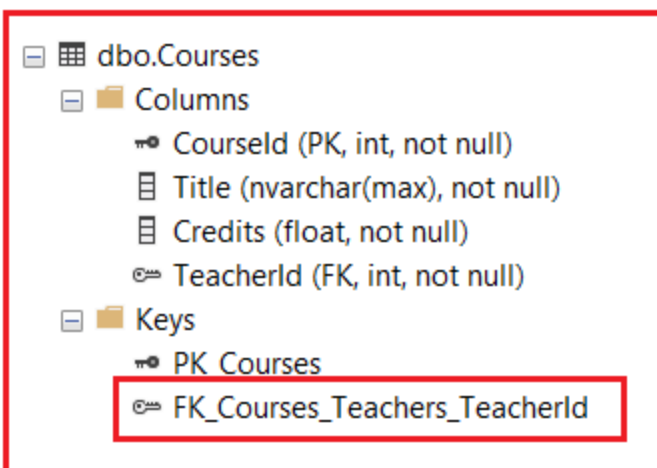
Primary Key (PK) Constraint Name:

By default, EF Core names the primary key constraint as **PK_<TableName>**. For example, if the table name is Students, the primary key constraint will be named **PK_Students**. For a better understanding, please look at the below image.



Foreign Key (FK) Constraint Name:

EF Core names foreign key constraints as **FK_<DependentTable>_<PrincipalTable>_<ForeignKeyColumn>**. For example, in the Courses table, it has a foreign key TeacherId referencing the Teachers table, so, the foreign key constraint will be named **FK_Courses_Teachers_TeacherId**.



Unique Constraint Name:

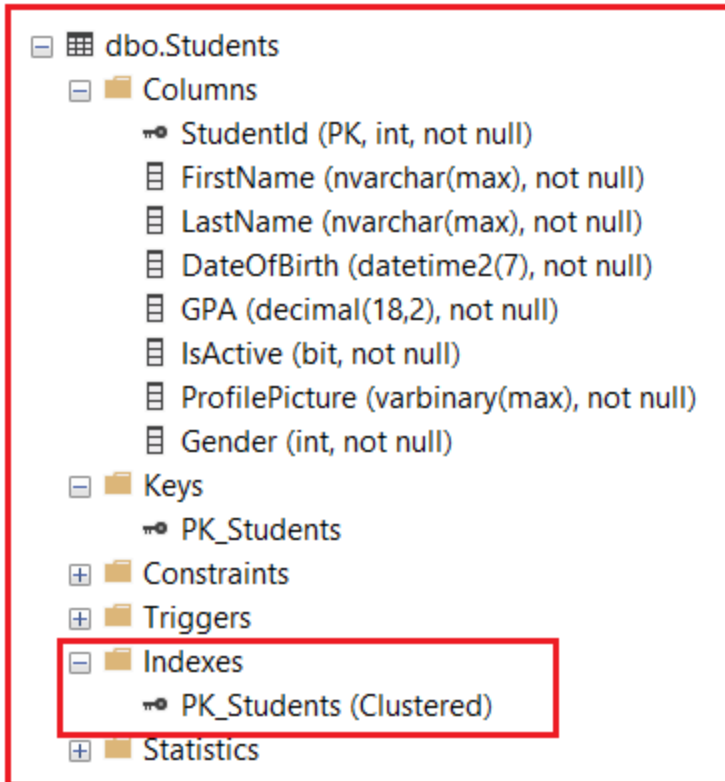
EF Core names unique constraints as **AK_<TableName>_<ColumnName>**. For example, If the Student table has a unique constraint on the Email column, the constraint name will be **AK_Student_Email**. We have not applied any Unique Constraint to any of our properties. We will understand this in our upcoming sessions.

Default Index Name in EF Core

EF Core automatically creates indexes to optimize database performance. By default, EF Core creates a clustered index on Primary key columns and a non-clustered index on foreign key columns. The default index name follows a specific convention in Entity Framework Core (EF Core).

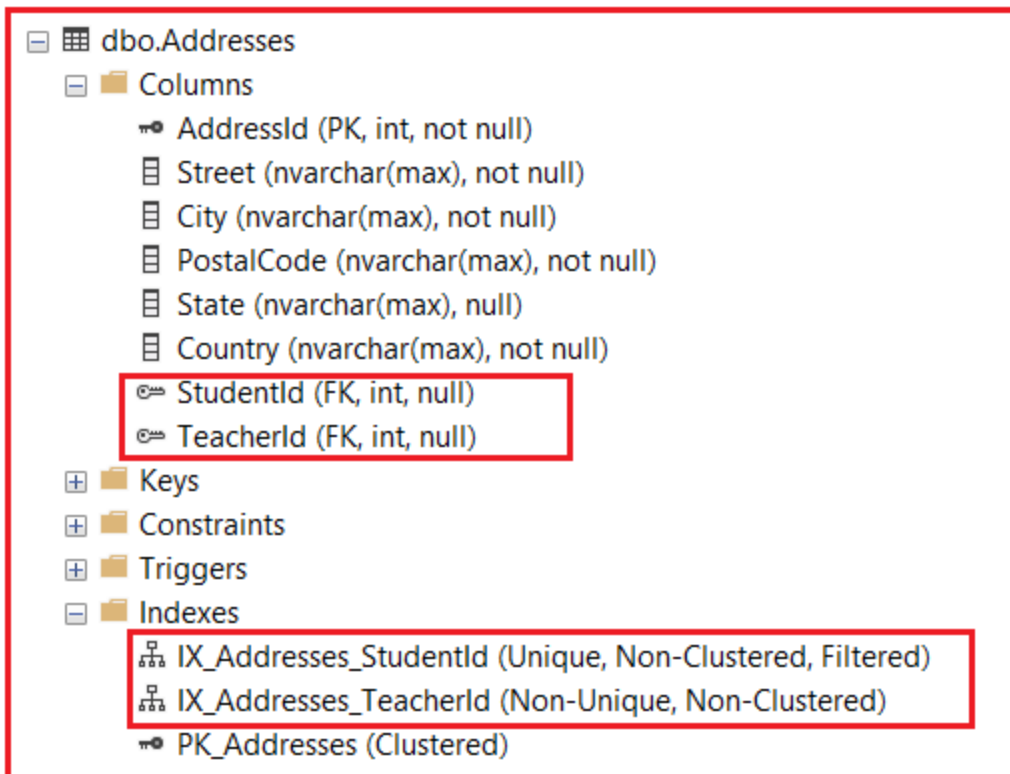
Primary Key Index:

When EF Core generates an index for the primary key column, it follows the primary key constraint naming convention (**PK_<TableName>**). For example, for a Students table with a primary key on the StudentId column, the index is implicitly created as part of the primary key constraint (**PK_Students**).



Foreign Key Index:

By default, EF Core also creates indexes on foreign key columns. The naming follows the same convention: **IX_<TableName>_<ForeignKeyColumn>**. For example, look at the following Addresses table. It has two foreign keys pointing to the Teachers and Students tables, and hence, it has two Non-Clustered Indexes.



Nullable Foreign Keys:

The `StudentId` and `TeacherId` foreign keys in the `Addresses` table are nullable. However, due to the relationships and default conventions, EF Core generates different types of indexes (Unique and Non-Unique) for these columns.

StudentId (Unique, Filtered): EF Core automatically creates a unique filtered index for `StudentId` because the relationship between `Address` and `Student` is one-to-one. In a one-to-one relationship, Each `Student` can have at most one associated `Address`.

EF Core enforces this by creating a unique index on the `StudentId` column, ensuring that no two rows in the `Addresses` table can reference the same `Student`. Since the `StudentId` column is nullable, EF Core applies a filtered index to ignore rows where `StudentId` is NULL. This ensures that only rows with a value in `StudentId` are checked for uniqueness.

TeacherId (Non-Unique): The relationship between the `Address` and the `Teacher` is one-to-many. This is because we have not applied any navigation property in the `Teacher` entity pointing to the `Address` entity, and hence, it is considered a one-to-many relationship.

In a one-to-many relationship, A `Teacher` can have multiple associated `Addresses`. Therefore, EF Core creates a non-unique index on the `TeacherId` column to optimize queries that search for rows with specific `TeacherId` values. A non-unique index allows duplicate `TeacherId` values in the `Addresses` table, as required for the one-to-many relationship.

Default Null and Not Null Columns in EF Core:

EF Core determines column nullability based on the C# property types:

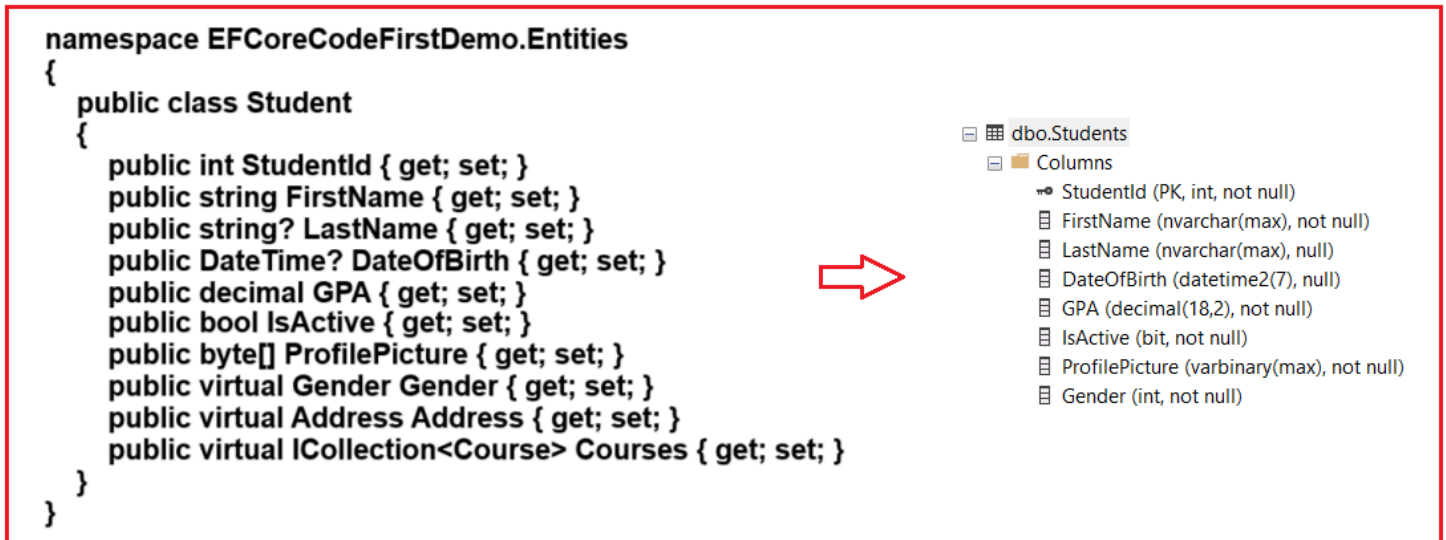
Value Types:

- **Non-Nullable:** `int`, `bool`, `decimal`, `DateTime`, etc., are mapped as NOT NULL.
- **Nullable:** `int?`, `bool?`, `decimal?`, `DateTime?`, etc., are mapped as NULL.

Reference Types:

- **By default**, string, byte[], and other reference types are treated as nullable (NULL).
- **Explicit Configuration:** Can be marked as required (NOT NULL) using Data Annotations ([Required]) or the Fluent API.

For a better understanding, please have a look at the image below, which shows the Student Entity and the corresponding Student database table.



Default Database Columns Order in EF Core:

EF Core orders columns in the database table based on their declaration order in the entity class, except the primary key, which is always placed first.

- **Primary Key:** If the primary key column is defined first in the class, it will appear first in the generated table. If it is not explicitly placed first, it will also appear first automatically in the database table.
- **Other Properties:** The remaining properties will appear in the database table in the order they are declared in the entity class.

For example, the following is our Address entity, where the AddressId is the fourth property on the list. If you look at the database table, you will see that the AddressId has been moved to the first position, as it is the primary key in the table.

```

public class Address
{
    0 references
    public string Street { get; set; }
    0 references
    public string City { get; set; }
    0 references
    public string PostalCode { get; set; }
    0 references
    public int AddressId { get; set; }
    0 references
    public string? State { get; set; }
    0 references
    public string Country { get; set; }
    0 references
    public int? StudentId { get; set; }
    0 references
    public virtual Student Student { get; set; }
    0 references
    public int? TeacherId { get; set; }
    0 references
    public virtual Teacher Teacher { get; set; }
}

```



```

dbo.Addresses
Columns
  AddressId (PK, int, not null)
  Street (nvarchar(max), not null)
  City (nvarchar(max), not null)
  PostalCode (nvarchar(max), not null)
  State (nvarchar(max), null)
  Country (nvarchar(max), not null)
  StudentId (FK, int, null)
  TeacherId (FK, int, null)

```

Default Column Data Type in EF Core

EF Core maps C# data types to SQL data types based on the database provider. For SQL Server, common mappings include:

C# Type	SQL Server Type	Description
int	int	4-byte integer (INT)
long	bigint	8-byte integer (BIGINT)
short	smallint	2-byte integer (SMALLINT)
byte	tinyint	1-byte integer (TINYINT)
bool	bit	Boolean value (BIT)
decimal	decimal(18, 2)	Decimal number with precision 18 and scale 2 (DECIMAL)
float	real	Single-precision floating-point number (REAL)
double	float	Double-precision floating-point number (FLOAT)
string	nvarchar(max)	Variable-length Unicode string (NVARCHAR(MAX))
char	nvarchar(1)	Single character (NVARCHAR(1))
DateTime	datetime2	Date and time (DATETIME2)
DateTimeOffset	datetimeoffset	Date and time with timezone offset (DATETIMEOFFSET)
TimeSpan	time	Time of day (TIME)
Guid	uniqueidentifier	Globally unique identifier (UNIQUEIDENTIFIER)
byte[]	varbinary(max)	Variable-length binary data (VARBINARY(MAX))
enum	int	Stored as integer (INT)

Default Cascade Behaviour in EF Core:

Cascade behaviors define how operations (like delete or update) are handled through related entities. EF Core sets default cascade behaviors based on the relationship's optionality and type.

Relationship (Required Dependent):

Suppose the dependent entity has a required relationship (i.e., the foreign key is NOT NULL). EF Core configures Cascade Delete behavior by default, i.e., Deleting the principal entity will automatically delete the related dependent entities.

In our example, there is a One-to-Many Required Relationship between Teacher and Courses. In this case, deleting a Teacher will also delete all related Courses. You can also verify the same. To do so, verify the Create table statement of the Courses database table. You should see the following: It is applying ON DELETE CASCADE behavior.

```

CREATE TABLE Courses(
    [CourseId] [int] IDENTITY(1,1) NOT NULL,
    [Title] [nvarchar](max) NOT NULL,
    [Credits] [float] NOT NULL,
    [TeacherId] [int] NOT NULL,
    CONSTRAINT [PK_Courses] PRIMARY KEY CLUSTERED
(
    [CourseId] ASC
)
) ON [PRIMARY] TEXTIMAGE_ON [PRIMARY]
GO

```

```

ALTER TABLE [dbo].[Courses] WITH CHECK ADD CONSTRAINT [FK_Courses_Teachers_TeacherId] FOREIGN KEY([TeacherId])
REFERENCES [dbo].[Teachers] ([TeacherId])
ON DELETE CASCADE
GO

```

Relationship (Optional Dependent):

If the dependent entity has an optional relationship (the foreign key is NULL), by default, EF Core configures Default Behavior as No Action. That means deleting the principal entity won't delete dependent entities; instead, it may set the foreign key to NULL or throw a constraint violation if NULL is not allowed on the Foreign Key column.

In our example, there is a One-to-One optional Relationship between the Teacher or Student and the Addresses. Deleting a Teacher or Student won't delete corresponding Address entries. Instead, Address.StudentId or Address.TeacherId is set to NULL. Now, if you verify the Create statement of the Addresses table, then you will not see any CASCADE DELETE behavior as shown below image:

```

CREATE TABLE Addresses(
    [AddressId] [int] IDENTITY(1,1) NOT NULL,
    [Street] [nvarchar](max) NOT NULL,
    [City] [nvarchar](max) NOT NULL,
    [PostalCode] [nvarchar](max) NOT NULL,
    [State] [nvarchar](max) NULL,
    [Country] [nvarchar](max) NOT NULL,
    [StudentId] [int] NULL,
    [TeacherId] [int] NULL,
)
GO

ALTER TABLE [dbo].[Addresses] WITH CHECK ADD CONSTRAINT [FK_Addresses_Students_StudentId]
FOREIGN KEY([StudentId])
REFERENCES [dbo].[Students] ([StudentId])
GO

ALTER TABLE [dbo].[Addresses] WITH CHECK ADD CONSTRAINT [FK_Addresses_Teachers_TeacherId]
FOREIGN KEY([TeacherId])
REFERENCES [dbo].[Teachers] ([TeacherId])
GO

```

Note: EF Core typically does not explicitly include an ON DELETE NO ACTION clause in the generated SQL migration scripts or database creation scripts because NO ACTION is the default behavior in SQL Server when no ON DELETE rule is specified.

One-to-One Relationship:

By default, a one-to-one relationship (where both entities have a primary key/foreign key relationship) is configured with cascade delete behavior if the foreign key is required. The related dependent entity is also deleted when the principal entity is deleted.

Many-to-Many Relationship:

In a many-to-many relationship, EF Core will automatically create a join table. Deleting an entity on either side of the relationship will cascade delete the entries from the join table but not the related entities on the other side of the relationship.

For example, there is many-to-many relationships between Students and Courses. Deleting a Student will also delete the associated entries from the join table (CourseStudent) but not the related Course entities from the Courses table. Similarly, deleting a Course entity from the Courses table will also delete related entities from the CourseStudent table but not from the Students table. Now, if you verify the CourseStudent create SQL statement, you will see the following:

```
CREATE TABLE CourseStudent(  
    [CoursesCourseId] [int] NOT NULL,  
    [StudentsStudentId] [int] NOT NULL,  
    CONSTRAINT [PK_CourseStudent] PRIMARY KEY CLUSTERED  
(  
    [CoursesCourseId] ASC,  
    [StudentsStudentId] ASC  
)  
) ON [PRIMARY]  
GO
```

```
ALTER TABLE [dbo].[CourseStudent] WITH CHECK ADD CONSTRAINT [FK_CourseStudent_Courses_CoursesCourseId]  
FOREIGN KEY([CoursesCourseId])  
REFERENCES [dbo].[Courses] ([CourseId])  
ON DELETE CASCADE  
GO
```

```
ALTER TABLE [dbo].[CourseStudent] WITH CHECK ADD CONSTRAINT [FK_CourseStudent_Students_StudentsStudentId]  
FOREIGN KEY([StudentsStudentId])  
REFERENCES [dbo].[Students] ([StudentId])  
ON DELETE CASCADE  
GO
```

Default Relationship in EF Core:

In Entity Framework Core (EF Core), the navigation properties determine the relationships between entities when generating the database tables. The type of relationship (one-to-one, one-to-many, or many-to-many) depends on how the navigation properties and foreign key properties are defined in the entity classes.

One-to-Many Relationship:

This is the most common type of relationship where one entity (the principal) can have many related entities (the dependent). EF Core configures a one-to-many relationship when we have a reference navigation property on one side and a collection navigation property on the other side.

In our example, we have an `ICollection<Course> Courses` collection navigation property in the Teacher entity, and in the Course entity, we have a `Teacher` reference navigation property. So, in this case, it will generate One-to-many relationships between Teachers and Courses, i.e., One Teacher can teach many Courses. The table Courses will have a `TeacherId` foreign key column pointing to the Teachers table.

One-to-One Relationship:

EF Core configures a one-to-one relationship when both entities have reference navigation properties and one entity contains the foreign key of the other. In our example, One Student can have one Address, and one Address belongs to one Student. The Addresses table will have a unique (one-to-one) StudentId foreign key pointing to the Students table.

Many-to-Many Relationship:

In EF Core 5.0 and later, many-to-many relationships can be configured without explicitly defining a join entity. EF Core will automatically create a join table to manage the relationship between the two entities. This relationship is established when both entities have a collection navigation property referencing each other.

In our example, there are many-to-many relationships between Students and Courses. One Student can enroll in many Courses, and multiple students can take a single course. EF Core automatically generates a join table called CourseStudent with foreign keys for StudentId and CourseId.

In the next article, I will discuss [Data Annotation Attributes in Entity Framework Core](#) with Examples. In this article, I try to explain **Default Conventions in Entity Framework (EF) Core** with Examples, and I hope you enjoyed this **Default Conventions in EF Core** with Examples article. Please provide your valuable feedback and suggestions for this article.